College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 8 Report: Edge Detection – Sobel, Prewitt and Canny

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab compares three classic edge detectors: Sobel, Prewitt and Canny. Sobel and Prewitt estimate the image gradient with small 3×3 kernels and return a gradient-magnitude map, while Canny goes further and produces thin, binary edges. The second task studies how Canny's two hysteresis thresholds change the result. Everything was implemented in Python with OpenCV, NumPy and Matplotlib.

2. Image Used

A new image is used instead of the one in the original lab notebook. It is stored in the Lab 8 folder:

- **Butterfly.jpg** – 800 × 532 colour photo of a monarch butterfly on clover. The wings have strong black veins, rows of small white spots and a dark border, and the background has leaves, stems and flower heads with many weaker edges.

3. Setup

OpenCV loads images in BGR order, so the image is converted to RGB for display and to grayscale for edge detection. A 5×5 Gaussian blur is applied first to reduce noise, because all three detectors are based on derivatives, which amplify noise.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Use a new image from the Lab 8 folder, not the original notebook image.
image = cv2.imread("Butterfly.jpg")

if image is None:
    raise FileNotFoundError("Butterfly.jpg could not be loaded.")

image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

4. Task 1 – Sobel, Prewitt and Canny

Sobel is computed with `cv2.Sobel` in the x and y directions. **Prewitt** has no built-in OpenCV function, so its two kernels are written out and applied with `cv2.filter2D`. For both, the gradient

magnitude $\sqrt{(G_x^2 + G_y^2)}$ is computed with `cv2.magnitude` in 32-bit float to avoid overflow, then scaled to 0–255 for display. **Canny** is applied with `cv2.Canny` using thresholds 50 and 150.

```
# Sobel Edge Detector
sobel_x = cv2.Sobel(blurred, cv2.CV_32F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(blurred, cv2.CV_32F, 0, 1, ksize=3)
sobel_magnitude = cv2.magnitude(sobel_x, sobel_y)
sobel_edges = cv2.normalize(sobel_magnitude, None, 0, 255,
                            cv2.NORM_MINMAX).astype(np.uint8)

# Prewitt Edge Detector
prewitt_x_kernel = np.array([[-1, 0, 1],
                             [-1, 0, 1],
                             [-1, 0, 1]], dtype=np.float32)
prewitt_y_kernel = np.array([[-1, -1, -1],
                             [ 0,  0,  0],
                             [ 1,  1,  1]], dtype=np.float32)
prewitt_x = cv2.filter2D(blurred, cv2.CV_32F, prewitt_x_kernel)
prewitt_y = cv2.filter2D(blurred, cv2.CV_32F, prewitt_y_kernel)
prewitt_magnitude = cv2.magnitude(prewitt_x, prewitt_y)
prewitt_edges = cv2.normalize(prewitt_magnitude, None, 0, 255,
                              cv2.NORM_MINMAX).astype(np.uint8)

# Canny Edge Detector
canny_edges = cv2.Canny(blurred, 50, 150)

# Display the original image and the three edge maps
images = [image, sobel_edges, prewitt_edges, canny_edges]
titles = ["Original Butterfly", "Sobel Edge Detector",
          "Prewitt Edge Detector", "Canny Edge Detector (50, 150)"]

fig, ax = plt.subplots(1, 4, figsize=(16, 5))
for axis, result, title in zip(ax, images, titles):
    if result.ndim == 2:
        axis.imshow(result, cmap="gray", vmin=0, vmax=255)
    else:
        axis.imshow(result)
    axis.set_title(title)
    axis.axis("off")

plt.tight_layout()
plt.show()
```

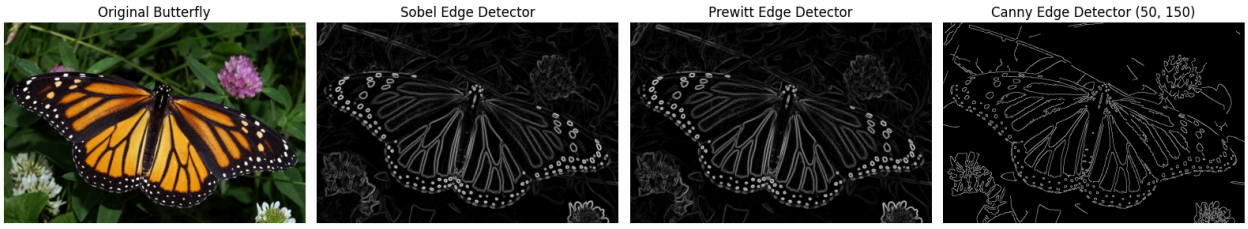


Figure 1: Original image and the edge maps from Sobel, Prewitt and Canny (50, 150).

Discussion: Sobel and Prewitt give almost the same result. Both show the wing veins, the white spots and the outline of the butterfly as bright lines, while the leaves and flowers in the background appear as fainter edges. The only difference between the kernels is that Sobel gives the centre row

or column a weight of 2 instead of 1, which adds a little smoothing across the edge. Its raw magnitude is therefore larger (about 1.34 times on average), but after normalising to 0–255 the two maps are nearly identical, with a correlation of about 0.9999. Both outputs are grayscale strength maps: the edges are several pixels wide and there is no clear decision about which pixels are edges and which are not.

Canny produces a binary map with edges only one pixel wide. It uses Sobel gradients internally, then keeps only local maxima along the gradient direction (non-maximum suppression) and links edges with two thresholds (hysteresis). As a result, the veins, spots and wing border are drawn as clean, continuous contours, and most of the soft texture in the background is removed. Only longer structures such as the stems and the flower heads remain. Canny gives the cleanest result for this image, while Sobel and Prewitt are better when the strength of an edge is needed rather than a yes/no decision.

5. Task 2 – Effect of Canny Thresholds

Canny is run on the same blurred image with three threshold pairs: low (30, 90), medium (50, 150) and high (100, 200). The number of edge pixels is counted with `np.count_nonzero`.

```
threshold_pairs = [(30, 90), (50, 150), (100, 200)]
canny_results = []

for low_threshold, high_threshold in threshold_pairs:
    edges = cv2.Canny(blurred, low_threshold, high_threshold)
    canny_results.append(edges)

fig, ax = plt.subplots(1, 3, figsize=(13, 4))

for axis, edges, (low_threshold, high_threshold) in zip(
        ax, canny_results, threshold_pairs):
    edge_count = np.count_nonzero(edges)
    axis.imshow(edges, cmap="gray", vmin=0, vmax=255)
    axis.set_title(f"Canny ({low_threshold}, {high_threshold})\n"
                   f"Edge pixels: {edge_count}")
    axis.axis("off")

plt.tight_layout()
plt.show()

for (low_threshold, high_threshold), edges in zip(threshold_pairs, canny_results):
    print(f"Thresholds ({low_threshold}, {high_threshold}): "
          f"{np.count_nonzero(edges)} edge pixels")
```

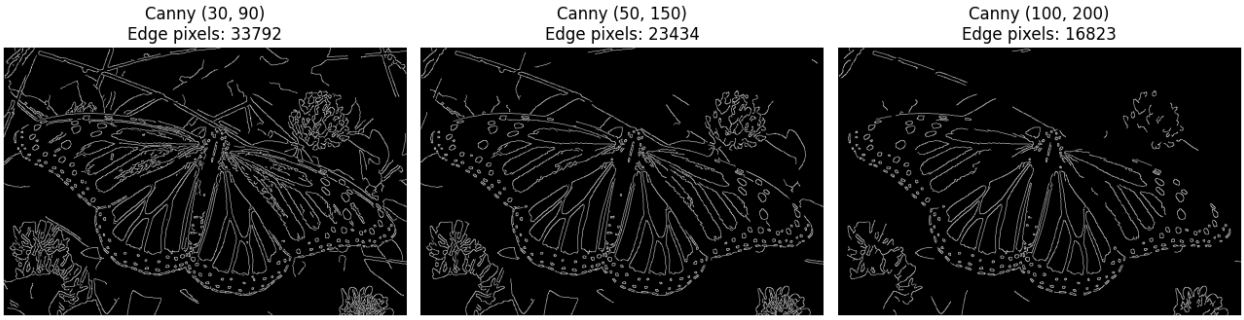


Figure 2: Canny edge maps for the threshold pairs (30, 90), (50, 150) and (100, 200).

Thresholds (low, high)	Edge pixels	% of image
(30, 90)	33 792	7.9 %
(50, 150)	23 434	5.5 %
(100, 200)	16 823	4.0 %

Discussion: In Canny, a pixel whose gradient is above the high threshold is a strong edge and is always kept. A pixel between the two thresholds is a weak edge and is kept only if it is connected to a strong edge. Pixels below the low threshold are removed. Raising the thresholds therefore removes edges: the count drops from 33 792 to 16 823 pixels, about half.

With (30, 90) almost every boundary is found, including the fine lines inside the wings, the leaf outlines and many short fragments in the background, so the result is detailed but cluttered. With (50, 150) most of the background clutter disappears, while the veins, the spots and the wing border remain complete; this is the best balance for this image. With (100, 200) only the strongest edges remain. The background is very clean, but some real details are lost and edges start to break up: parts of the inner veins and some of the spots along the wing border become incomplete. Because of hysteresis, the low threshold mainly controls how far edges are extended and therefore their continuity, while the high threshold controls which edges are detected at all.

6. Conclusion

Sobel and Prewitt are simple gradient operators that return how strong the edge is at each pixel. They give almost identical results, with Sobel being slightly more resistant to noise because of its centre weighting. Canny builds on the Sobel gradient and adds non-maximum suppression and hysteresis thresholding, which gives thin, connected, binary edges and the cleanest result on the butterfly image. Its quality depends on the threshold pair: low thresholds keep detail but also noise, high thresholds give a clean map but break weak edges. For this image, thresholds of about (50, 150) with a 5×5 Gaussian blur gave the best trade-off.

7. References

Sandipan Dey, *Hands-On Image Processing with Python*, Packt.

OpenCV: Canny Edge Detection – docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

OpenCV: Image Gradients – docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html